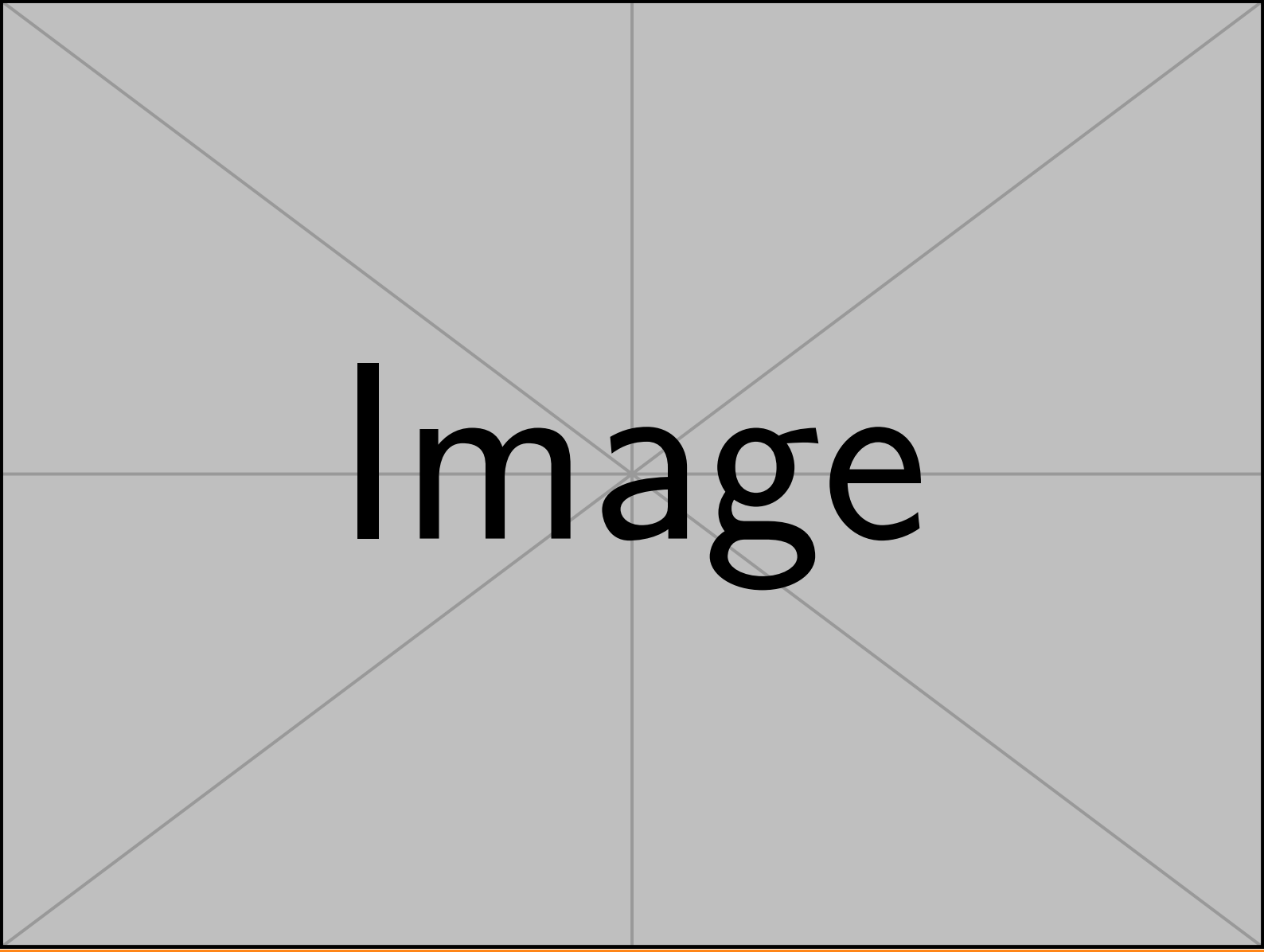


Slip User Manual

A Minimal HDL DSL Compiler

Author: Slip Project

Date: May 2026

Version: 1.0

Contents

Chapter 1 Introduction

Slip is a minimal hardware description language (DSL) that compiles `.slip` source files into synthesizable SystemVerilog. It is designed as syntactic sugar over SystemVerilog, providing a streamlined syntax for common hardware design patterns while generating clean, human-readable, IEEE 1800-2017 compliant output.

1.1 Design Philosophy

- **Minimal syntax** — port directions, signal types, and instance wiring are inferred from context.
- **Direct mapping** — every Slip construct maps 1:1 to a SystemVerilog equivalent.
- **Compile-time metaprogramming** — `` `for `` loops and `` `if `` conditionals are fully expanded before code generation, with SystemVerilog constant evaluation.
- **Validated output** — generated SystemVerilog is checked against the IEEE 1800-2017 standard via slang.

Chapter 2 Installation

2.1 Prerequisites

- A C++20 compiler and CMake $\geq$ 3.24
- PCRE2 8-bit development libraries
- slang 10, installed or fetched by CMake from the pinned source revision
- Python only when building slang's generated C++ headers from source

The Slip executable does not require Python or pyslang at runtime. The Python implementation is maintained on the `python-reference` branch.

2.2 Build and Install

```
git clone https://github.com/himingway/slip.git
cd slip
cmake --preset release
cmake --build --preset release --parallel 2
ctest --preset release
cmake --install build/release --prefix "$HOME/.local"
```

Add `$HOME/.local/bin` to `PATH` to use `slip`. Offline builds can use `CMAKE_PREFIX_PATH` and `SLIP_FETCH_SLANG=OFF`; see `docs/development.md`. Verilator enables the independent RTL behavioral test.

Chapter 3 Command Line Interface

3.1 `slip build` — Compile to SystemVerilog

```
slip build <file.slip>                    # output to ./build/
slip build -o <out_dir> <file.slip>       # custom output directory
slip build -ip <ip_dir> <file.slip>       # include external IP directory (recursive)
slip build -ip <dir1> -ip <dir2> <file.slip> # multiple IP directories
slip build -f <file.f> <file.slip>        # use VCS-format filelist
slip build -f <file.f> -ip <dir> <file.slip> # filelist + directory (combined)
```

Each module in the `.slip` file generates a separate `.sv` file in the output directory.

3.2 `slip check` — Validate without generating output

```
slip check <file.slip>                    # validate only
slip check -ip <ip_dir> <file.slip>       # with IP directory
slip check -f <file.f> <file.slip>        # with filelist
```

Runs the full compilation pipeline but discards the output. Generated modules and external IP are validated together, sharing the same IP preprocessing context. All modules are validated before any build output is written.

Use `--strict` with either command in CI to require explicit signal and port declarations and reject assignment or port width mismatches. Default mode continues to support annotated implicit signals.

Chapter 4 Language Overview

A Slip source file contains one or more module declarations. Comments use `//` (line) or `/* */` (block) syntax.

```
// This is a comment
module AND_gate (a, b, y) {
    logic a;
    logic b;
    logic y;
    assign y = a & b;
}
```

Generates:

```
module AND_gate (
    input logic a,
    input logic b,
    output logic y
);

    assign y = a & b;

endmodule
```

4.1 Key Differences from SystemVerilog

- No `input`/`output` keywords — port directions are inferred.
- No `wire`/`reg` — everything is `logic`.
- Blocking assignment in `seq` blocks is automatically converted to non-blocking.
- Compile-time metaprogramming via `` `for `` and `` `if ``.
- Template identifiers embed loop variables in signal names (e.g., `data_`i`).
- `include "file.slip"` directive for reusing `defun` functions and modules across files.
- Generated SV annotated with `// implicit`, `// no width`, `// width from X.Y` for easy review.

Chapter 5 Module Declaration

5.1 Basic Syntax

```
module <name> (port1, port2, ...) {
    // body
}
```

5.2 With Parameters

```
module <name> #(param W = 8, param DEPTH = 16) (port1, port2, ...) {
    // body
}
```

5.3 Implicit Ports

If the port list is omitted entirely, all identifiers used in the module body that are not declared internally become ports:

```
module passthrough {
    assign out = in;
}
```

Generates:

```
module passthrough (
    input logic in,
    output logic out
);

    assign out = in;

endmodule
```

Port directions are inferred from usage (see Chapter **??**).

Chapter 6 Parameters and Localparams

6.1 Parameters (`param`)

Parameters are declared in the module header and can be overridden during instantiation unless they control Slip compile-time expansion:

```
module counter #(param WIDTH = 8, param MAX = 255) (clk, rst_n, count) {
    logic clk;
    logic rst_n;
    logic [WIDTH-1:0] count;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            count <= 0;
        } else if (count == MAX) {
            count <= 0;
        } else {
            count <= count + 1;
        }
    }
}
```

Generates:

```
module counter #(
    parameter WIDTH = 8,
    parameter MAX = 255
) (
    input logic clk,
    input logic rst_n,
    output logic [WIDTH - 1:0] count
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count <= 0;
        end else begin
            if (count == MAX) begin
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end

endmodule
```

Override at instantiation:

```
module top (clk, rst_n, count) {
    logic clk;
    logic rst_n;
    logic [15:0] count;
    counter #(.WIDTH(16), .MAX(65535)) u_cnt { .clk, .rst_n, .count };
}
```

Generates:

```
module top (
    input logic clk,
    input logic rst_n,
    input logic [15:0] count
);

    counter #(
        .WIDTH(16),
        .MAX(65535)
    ) u_cnt (
        .clk(clk),
        .rst_n(rst_n),
        .count(count)
    );

endmodule
```

6.2 Localparams (localparam)

Localparams are declared in the module body and **cannot** be overridden during instantiation:

```
module fsm (clk, state) {
    logic clk;
    logic [1:0] state;
    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;
}
```

Generates:

```
module fsm (
    input logic clk,
    input logic [1:0] state
);

    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;

endmodule
```

Attempting to override a localparam in an instance raises a semantic error:

```
fsm #(.IDLE(3)) u1 { .clk, .state }; // ERROR
```

Localparams can be used in signal widths, `` `for `` bounds, and `` `if `` conditions.

Chapter 7 Signals

7.1 Explicit Declaration

```
logic [7:0] data;        // 8-bit signal
logic flag;              // 1-bit signal
signed [15:0] val;       // signed 16-bit
logic [7:0] mem [0:15]; // array of 16 x 8-bit
logic [WIDTH-1:0] bus;   // parameterized width
```

The `logic` keyword is optional for bare declarations:

```
data;                    // equivalent to: logic data;
```

7.2 Implicit Declaration

Signals that are referenced but not declared are automatically created as `logic` signals:

```
module example (a, b) {
    logic a;
    logic b;
    assign y = a & b; // 'y' is implicitly declared as logic
}
```

Generates:

```
module example (
    input logic a,
    input logic b
);

    logic y;

    assign y = a & b;

endmodule
```

Implicit signals used in instance port connections automatically inherit the target port's width, annotated with `// width from X.Y` in the generated SV. Other implicit signals default to 1-bit and are annotated with `// implicit, no width` for easy review.

Chapter 8 Port Inference

Slip infers port directions automatically from signal usage within the module body:

Usage Pattern	Inferred Direction
Only appears in expressions (right-hand side)	`input`
Only appears as assignment target (left-hand side)	`output`
Both read and driven	`inout`

8.1 Example

```
module adder (a, b, sum) {
    logic a;
    logic b;
    logic sum;
    assign sum = a + b;
}
```

Generates:

```
module adder (
    input logic a,
    input logic b,
    output logic sum
);

    assign sum = a + b;

endmodule
```

Chapter 9 Assignments

9.1 Continuous Assignment

At module level, both forms generate `assign` statements in SystemVerilog:

```
assign y = a & b;        // explicit 'assign' keyword
y = a + b;               // bare assignment (also generates 'assign')
```

Both generate continuous assignments:

```
assign y = a & b;
assign z = a + b;
```

9.2 Procedural Assignment

Inside `seq` and `comb` blocks, assignments become procedural:

```
seq (clk) {
    q <= d;              // non-blocking (sequential)
}

comb {
    y = sel ? a : b;     // blocking (combinational)
}
```

Generates:

```
    always_ff @(posedge clk) begin
        q <= d;
    end

    always_comb begin
        y = sel ? a : b;
    end
```

Remark The `assign` keyword must **not** be used inside `seq` or `comb` blocks. It is reserved for module-level continuous assignments only.

Chapter 10 Sequential Logic (`seq`)

The `seq` block generates an `always_ff` block with the specified clock and optional asynchronous reset.

10.1 Basic Clock

```
seq (clk) {
    q <= d;
}
```

Generates:

```
    always_ff @(posedge clk) begin
        q <= d;
    end
```

10.2 Clock with Asynchronous Reset

```
seq (clk, neg: rst_n) {
    if (!rst_n) {
        q <= 0;
    } else {
        q <= d;
    }
}
```

Generates:

```
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 0;
        end else begin
            q <= d;
        end
    end
```

Reset polarity options:

- `neg:` `rst_n` — active-low reset (`negedge rst_n`)
- `pos:` `rst` — active-high reset (`posedge rst`)

10.3 Automatic Blocking-to-Nonblocking Conversion

All blocking assignments (=) inside `seq` blocks are automatically converted to non-blocking (<=). This prevents common sequential logic mistakes:

```
seq (clk) {
    q = d;  // automatically becomes q <= d;
}
```

Generates (blocking = converted to non-blocking <=):

```
    always_ff @(posedge clk) begin
        q <= d;
    end
```

The conversion is unconditional. A warning is emitted for the assignments whose meaning the conversion changes: those that read their own previous value (a self-referencing statement such as `for (...) { sum = sum + b; }`, or a variable written and then read again on the same path). Under non-blocking semantics the read observes the pre-clock value, so such code should be restructured (or moved to a `comb` block) if the updated value was intended. Standard register idioms — shift registers over arrays, state machines, gated counters — convert silently.

Chapter 11 Combinational Logic (`comb`)

The `comb` block generates an `always_comb` block. Assignments inside remain blocking:

```
comb {
    if (sel) {
        y = a;
    } else {
        y = b;
    }
}
```

Generates:

```
    always_comb begin
        if (sel) begin
            y = a;
        end else begin
            y = b;
        end
    end
```

If the combinational block infers a latch (e.g., incomplete assignment in an `if` without `else`), Slip emits `always_latch` without a sensitivity list:

```
always_latch begin
    if (en) begin
        q <= d;
    end
end
```

Chapter 12 Initial Blocks

The `initial` block generates an `initial` block. It is primarily used in testbenches for initialization sequences. Assignments inside remain blocking:

```
initial {
    clk = 0;
    rst_n = 0;
}
```

Generates:

```
    initial begin
        clk = 0;
        rst_n = 0;
    end
```

Chapter 13 Case Statements

Slip supports `case`, `casez`, and `casex` statements with a brace-based syntax.

13.1 Basic Case

```
case (sel) {
    2'b00: y = a;
    2'b01: y = b;
    2'b10, 2'b11: y = c;
    default: y = 0;
}
```

Generates:

```
    always_comb begin
        case (sel)
            2'b00: begin
                y = a;
            end
            2'b01: begin
                y = b;
            end
            2'b10, 2'b11: begin
                y = c;
            end
            default: begin
                y = 0;
            end
        endcase
    end
```

13.2 Casez with Wildcards

`casez` treats `?` as don't-care bits:

```
casez (opcode) {
    4'b1???: y = a;
    4'b01??: y = b;
    4'b001?: y = c;
    4'b0001: y = d;
    default: y = 0;
}
```

Generates:

```
    always_comb begin
```

```
        casez (opcode)
            4'b1???: begin
                y = a;
            end
            4'b01??: begin
                y = b;
            end
            4'b001?: begin
                y = c;
            end
            4'b0001: begin
                y = d;
            end
            default: begin
                y = 0;
            end
        endcase
    end
```

13.3 Multi-Pattern Items

A single case item can match multiple patterns, separated by commas:

```
localparam IDLE = 0;
localparam WAIT = 1;
localparam RUN = 2;

comb {
    case (state) {
        IDLE, WAIT: y = 0;
        RUN: y = 1;
        default: y = 0;
    }
}
```

Generates (assuming `IDLE=0`, `WAIT=1`, `RUN=2`):

```
localparam IDLE = 0;
localparam WAIT = 1;
localparam RUN = 2;

    always_comb begin
        case (state)
            IDLE, WAIT: begin
                y = 0;
            end
            RUN: begin
                y = 1;
            end
            default: begin
                y = 0;
```

```
            end
        endcase
    end
```

Chapter 14 Inside Operator

The `inside` operator tests set membership, following SystemVerilog 2009:

```
if (state inside {IDLE, RUN, DONE}) begin
    // state is one of IDLE, RUN, DONE
end
```

The right-hand side is a set literal `{v1, v2, ...}`. At compile time, if all values are constants, the expression evaluates to 0 or 1.

```
localparam IDLE = 0;
localparam RUN = 1;
localparam DONE = 2;

comb {
    if (state inside {IDLE, RUN, DONE}) {
        active = 1;
    } else {
        active = 0;
    }
}
```

Generates:

```
    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;

    logic active;

    always_comb begin
        if (state inside {IDLE, RUN, DONE}) begin
            active = 1;
        end else begin
            active = 0;
        end
    end
```

Chapter 15 Compound Assignments

Slip supports all SystemVerilog compound assignment operators. They are desugared at parse time:

Operator	Desugars To	Example
`+=`	`a = a + b`	`i += 1`
`-=`	`a = a - b`	`count -= 1`
`*=`	`a = a * b`	`val *= 2`
`/=`	`a = a / b`	`val /= 2`
`%=`	`a = a % b`	`val %= N`
`&=`	`a = a & b`	`flags &= MASK`
`|=`	`a = a | b`	`flags |= BIT`
`^=`	`a = a ^ b`	`flags ^= BIT`
`<<=`	`a = a << b`	`val <<= 2`
`>>=`	`a = a >> b`	`val >>= 2`
`<<<=`	`a = a <<< b`	`val <<<= 1`
`>>>=`	`a = a >>> b`	`val >>>= 1`

Compound assignments are valid in all contexts where regular assignments are used (module-level, `comb`, `seq`, `initial`).

15.1 Example

```
module m (clk, val, delta) {
    logic clk;
    logic [7:0] val;
    logic [7:0] delta;

    seq (clk) {
        val += delta;
        val -= 1;
    }
}
```

Generates (compound assignments desugared, blocking converted to non-blocking):

```
module m (
    input logic clk,
    output logic [7:0] val,
    input logic [7:0] delta
);

    always_ff @(posedge clk) begin
        val <= val + delta;
        val <= val - 1;
    end
```

```
endmodule
```

Chapter 16 Control Flow

16.1 If / Else

```
if (condition) begin
    // ...
end else begin
    // ...
end
```

16.2 For Loop

Procedural `for` loops generate SystemVerilog `for` statements. They are valid inside `seq` and `comb` blocks:

```
seq (clk, neg: rst_n) {
    if (!rst_n) {
        for (i = 0; i < 4; i = i + 1) {
            reg_file[i] <= 0;
        }
    }
}
```

Generates:

```
    logic [7:0] reg_file [0:3];

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            for (int i = 0; i < 4; i = i + 1) begin
                reg_file[i] <= 0;
            end
        end
    end
```

Loop variables (e.g., `i`) are excluded from implicit signal creation — they are treated as loop-local integers.

16.3 For-Loop Step Variants

The step clause supports three forms:

```
for (i = 0; i < N; i = i + 1) { ... } // standard
for (i = 0; i < N; i += 1) { ... }  // compound assignment
for (i = 0; i < N; i++)    { ... }  // increment
```

Compound assignments (`+=`, `-=`, `*=`, etc.) are desugared at parse time: `i += 1` becomes `i = i + 1`. All three

forms generate the same SystemVerilog output:

```
for (int i = 0; i < N; i = i + 1) begin
    // ...
end
```

16.4 Restrictions

- `while` loops are not supported (combinational while loops are unsafe in synthesis).

Chapter 17 Include Directive

The `include` directive imports `defun` function definitions and module declarations from another `.slip` file. It must appear at the top level, outside module bodies.

17.1 Basic Usage

```
include "helpers.slip";
include "utils/common.slip";
```

Included files are lexed, parsed, and their contents merged into the compilation unit. Relative paths are resolved from the directory of the including file.

17.2 Example: Sharing defun Libraries

File `helpers.slip`:

```
defun prefix(s):
    return "bus_" + s

defun inc(n):
    return n + 1
```

File `design.slip`:

```
include "helpers.slip";

module top (clk) {
    logic clk;
    inner u1 {
        .clk,
        "data_(.*)" => "$prefix(\1)",
        "ch_(.*)" => "$inc(\1)"
    };
}
```

17.3 Recursive Includes

Include directives in included files are also resolved. The same file is never included twice (deduplicated by absolute path).

17.4 Restrictions

- `include` must appear at the top level — not inside module bodies.
- The included path must be a string literal. Variable-based paths are not supported.
- Circular includes are detected and silently skipped.

Chapter 18 Module Instantiation

18.1 Basic Instantiation

```
module_name instance_name {
    .port1(signal1),
    .port2(signal2)
};
```

18.2 Same-Name Shorthand

When the port name matches the signal name, the signal expression can be omitted:

```
inner u1 { .clk, .rst_n, .data_in };
// Equivalent to: .clk(clk), .rst_n(rst_n), .data_in(data_in)
```

18.3 Parameter Override

```
inner #(.WIDTH(16), .DEPTH(32)) u1 { .clk, .data_in, .data_out };
```

18.4 Dangling Ports (_)

Use _ to leave a port intentionally unconnected:

```
inner u1 { .clk, .rst_n(_), .data_in };
```

In the generated SystemVerilog, the `.rst_n()` connection is omitted entirely.

18.5 Regex Port Mapping

Use regex patterns to map groups of ports to signals:

```
child u1 {
    .clk,
    "data_(.*)" => "bus_\1"
};
```

This maps `data_0` → `bus_0`, `data_1` → `bus_1`, etc. Signal widths are inferred from the target module's port declarations.

Regex connections are processed in order and can be mixed with explicit connections. The priority order is:

1. Explicit connections (highest priority)
2. Same-name shorthand
3. Regex patterns (first matching rule wins)

18.5.1 Regex Functions

Use `$func(...)` syntax to transform captured groups in replacements:

```
child u1 {
    .clk,
    "data_(.*)" => "bus_$upper(\1)", // uppercase
    "ch_(.*)"  => "port_$add(\1, 1)" // arithmetic
};
```

Built-in functions:

- **String**: `$upper`, `$lower`, `$reverse`, `$substr`, `$replace`, `$concat`
- **Arithmetic**: `$add`, `$sub`, `$mul`, `$div`, `$mod`
- **Bit**: `$bit_reverse`, `$bit_select`

The arithmetic functions require a numeric capture (a port named `ch_b` cannot be incremented; use a string function for non-numeric names). The failure is reported as a located error naming the port and instance.

Multiple functions and prefix/suffix text are supported:

```
"port_(.*)" => "pre_$upper(\1)_post" // prefix + function + suffix
"a_(.*)_b_(.*)" => "$upper(\1)_$lower(\2)" // multiple functions
```

18.5.2 Native Custom Functions (defun)

Define custom functions directly in Slip using Python-style syntax:

```
defun prefix(s):
    return "bus_" + s

defun to_upper(s):
    return s.upper()

defun inc(n):
    return n + 1

module top (clk) {
    logic clk;
    inner u1 {
        .clk,
        "data_(.*)" => "$prefix(\1)",
        "name_(.*)" => "$to_upper(\1)",
        "ch_(.*)" => "$inc(\1)"
    };
}
```

The C++ interpreter supports a restricted Python-style expression language, including:

- String concatenation: `a + b`
- String methods: `.upper()`, `.lower()`, `.reverse()`, `.replace()`, `.strip()`, `.split()`, `.startswith()`, etc.
- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Method chaining: `s.upper().reverse()`
- Built-in functions: `len()`, `int()`, `str()`, `range()`, etc.
- Indexing and slicing: `s[0]`, `s[1:3]`, `s[3:]`, `s[:3]`, `s[-1]`
- Ternary expressions: `x if cond else y`

18.5.2.1 Examples

Replace characters in port names:

```
defun fix_names(s):
    return s.replace("_", "-")

module inner (data_in, data_out) {
    logic data_in;
    logic data_out;
    assign data_out = data_in;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "data_(.*)" => "bus_$fix_names(\1)" };
    // data_in -> bus_data-in, data_out -> bus_data-out
}
```

Extract part of port name using slicing:

```
defun shorten(s):
    return s[:4] // take first 4 characters

module inner (channel_a, channel_b) {
    logic channel_a;
    logic channel_b;
    assign channel_b = channel_a;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "channel_(.*)" => "$shorten(\1)" };
    // channel_a -> chan, channel_b -> chan
}
```

Combine methods with chaining:

```
defun clean_upper(s):
    return s.strip().upper().replace("_", "")
```

```
module inner (sig_in, sig_out) {
    logic sig_in;
    logic sig_out;
    assign sig_out = sig_in;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "sig_(.*)" => "port_$clean_upper(\1)" };
    // sig_in -> port_SIGIN, sig_out -> port_SIGOUT
}
```

Conditional logic with `startswith()`:

```
defun prefix_by_type(s):
    return "axi_" + s if s.startswith("ar") else "apb_" + s

module inner (araddr, awaddr, wdata) {
    logic araddr;
    logic awaddr;
    logic wdata;
    assign wdata = araddr;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "(.*)" => "$prefix_by_type(\1)" };
    // araddr -> axi_araddr, awaddr -> apb_awaddr, wdata -> apb_wdata
}
```

Functions can be defined anywhere at the top level (before or after modules). All `defun` functions are registered before instance resolution.

18.6 Port Connection Constants

Port connection expressions that are constants must follow these rules:

- **Allowed**: width-prefixed literals (e.g., `1'b0`, `8'hFF`) and fill constants (`'0`, `'1`).
- **Forbidden**: bare integers (e.g., `0`, `42`).

```
// WRONG -- bare integer in port connection
inner u1 { .data_in(0) };

// OK -- width-prefixed
inner u1 { .data_in(1'b0) };

// OK -- fill constant (width determined by target port)
inner u1 { .data_in('0) };
```

The fill constants `'0` and `'1` are SystemVerilog native adaptive-width constants. Slip passes them through

directly; the downstream tool determines their width from the target port.

Chapter 19 Metaprogramming

Slip provides compile-time `‘for` loops and `‘if` conditionals. These are fully expanded before code generation.

19.1 `‘for` Loop

Syntax uses backtick-prefixed keywords and loop variables:

```
‘for (‘i = 0; ‘i< 4; ‘i= ‘i + 1) {
    logic [7:0] data_‘i;
    assign data_‘i = ‘i;
}
```

After expansion, this becomes:

```
logic [7:0] data_0;
logic [7:0] data_1;
logic [7:0] data_2;
logic [7:0] data_3;
assign data_0 = 0;
assign data_1 = 1;
assign data_2 = 2;
assign data_3 = 3;
```

19.1.1 Template Identifiers

Loop variables can appear inside identifier names using backtick syntax. The lexer merges the surrounding identifier text with the backtick variable into a single token:

```
‘for (‘i = 0; ‘i< 3; ‘i= ‘i + 1) {
    assign out_‘i = in_‘i;
}
```

Generates:

```
    logic out_0;
    logic in_0;
    logic out_1;
    logic in_1;
    logic out_2;
    logic in_2;

    assign out_0 = in_0;
    assign out_1 = in_1;
    assign out_2 = in_2;
```

19.1.2 Nested Loops

```
`for (`i = 0; `i< 2; `i= `i + 1) {
    `for (`j = 0; `j< 2; `j= `j + 1) {
        assign result_`i_`j = `i+ `j;
    }
}
```

Generates:

```
    logic result_0_0;
    logic result_0_1;
    logic result_1_0;
    logic result_1_1;

    assign result_0_0 = 0;
    assign result_0_1 = 1;
    assign result_1_0 = 1;
    assign result_1_1 = 2;
```

Template identifiers with multiple backtick variables (e.g., `a`i_`j`) are fully supported. The lexer merges them into a single token in a single pass.

19.1.3 Constant Folding

Loop variables are substituted into hardware expressions without changing literal widths, signedness, four-state values, or assignment context. Hardware arithmetic is left to SystemVerilog. Pure decimal constants in index and dimension expressions can be folded; algebraic identities such as replacing an unknown value multiplied by zero are not applied.

Compile-time controls are evaluated by slang with SystemVerilog semantics. Parameters used directly or indirectly by these controls are fixed at their generation-time defaults. Slip rejects overrides of these parameters, and generated elaboration checks reject different values supplied by downstream SystemVerilog. Change the source configuration and rebuild to change such a parameter. Ordinary RTL parameters remain overridable.

Example — bit reversal with parameterized width:

```
module bit_reverse #(param W = 8) (din, dout) {
    logic [W-1:0] din;
    logic [W-1:0] dout;

    comb {
        `for (`i = 0; `i< W; `i= `i + 1) {
            dout[W - 1 - `i] = din[`i];
        }
    }
}
```

When `W=8` and `` `i=0 ``, substitution produces `dout[W - 1 - 0] = din[0]`. Expressions containing `W` stay symbolic and are evaluated by the downstream tool.

19.1.4 Compile-Time Expression Evaluation

`for init, bound, and step expressions, as well as `if conditions, must be evaluable to integer constants at compile time. The evaluation engine supports:

- Integer literals and `param`/`localparam` references
- Arithmetic: +, -, *, /, %, **
- Relational: <, >, <=, >=, ==, !=, ===, !==
- Logical: &&, ||
- Bitwise: &, |, ^, ~, <<, >>, <<<, >>>
- Unary prefix: -, !, ~
- Parenthesized sub-expressions

19.2 `if Conditional

```
`if (ENABLE_DEBUG) {
    logic [7:0] debug_out;
    assign debug_out = internal_signal;
}
`else {
    assign debug_out = 0;
}
```

When `ENABLE_DEBUG` evaluates to non-zero, generates:

```
    logic [7:0] debug_out;
    assign debug_out = internal_signal;
```

When `ENABLE_DEBUG` evaluates to zero, generates:

```
assign debug_out = 0;
```

The condition must be a compile-time constant expression (parameter, localparam, or integer literal expression).

19.3 Restrictions

- Loop variables (`` `i ``) cannot be assigned within the loop body — they are controlled by the expansion engine.
- `` `for `` / `` `if `` conditions must be evaluable at compile time.
- The step variable must match the loop variable (e.g., `` `i = `i + 1 ``).
- A default maximum of 10,000 loop iterations across each module; repeated loop values are diagnosed immediately.

Chapter 20 Expressions and Operators

20.1 Operator Precedence

From highest to lowest (following IEEE 1800-2017):

Priority	Operators	Associativity
1 (highest)	`[]`, `[ : ]`, function call, `N'(expr)`	Left
2	`+`, `-`, `!`, `~` (unary); `signed'`, `unsigned'`	Right
3	`**`	Right
4	`*`, `/`, `%`	Left
5	`+`, `-` (binary)	Left
6	`<<`, `>>`, `<<<`, `>>>`	Left
7	`<`, `>`, `<=`, `>=`	Left
8	`==`, `!=`, `===`, `!==`	Left
9	`&`	Left
10	`^`	Left
11	`|`	Left
12	`&&`	Left
13	`||`	Left
14	`?:` (ternary)	Right
15 (lowest)	`inside` (set membership)	Left

20.2 Bit Selection and Part Select

```
data[3]        // bit select
data[7:0]      // part select
data[i]        // variable index
```

20.3 Concatenation and Replication

```
{a, b, c}      // concatenation
{4{1'b1}}      // replication: 4 copies of 1'b1
```

Nested replications are supported: `{4{{2{1'b0}}}}` produces 8 bits of zero.

20.4 Type Casting

```
signed'(expr)  // signed cast
unsigned'(expr) // unsigned cast
8'(expr)       // width cast (truncate or zero-extend to 8 bits)
```

Both `signed` and `unsigned` without a following `'` raise a syntax error.

20.5 Ternary Operator

```
sel ? a : b
```

Right-associative: `a ? b : c ? d : e` is parsed as `a ? b : (c ? d : e)`.

Chapter 21 Special Constants

21.1 Fill Constants

```
'0   // all bits zero (width determined by context)
'1   // all bits one (width determined by context)
```

These are SystemVerilog native fill constants. Their width is automatically determined by the connected port or assignment target. Slip passes them through without width inference.

21.2 Width-Prefixed Literals

```
1'b0        // 1-bit binary zero
1'b1        // 1-bit binary one
8'hFF       // 8-bit hexadecimal
4'b1010     // 4-bit binary
16'd255     // 16-bit decimal
8'o377      // 8-bit octal
```

Digits are validated against the specified radix. For example, `4'bABCD` and `4'o888` are rejected. The four-state values `x` and `z` are accepted in any radix.

21.3 Bare Integers

```
0           // bare integer (OK in assignments and expressions)
42          // bare integer
```

Bare integers are **not allowed** in port connections — use width-prefixed literals or fill constants instead.

Chapter 22 IP Integration

Slip can instantiate external SystemVerilog modules by providing IP directories or VCS-format filelists:

```
slip build -ip ./ip_cores -ip ./vendor_lib design.slip
slip build -f ip.f design.slip
slip build -f ip.f -ip ./ip_cores design.slip
```

22.1 How It Works

1. Slip collects IP sources from filelists (`-f`) and IP directories (`-ip`).
2. IP directories are scanned recursively for `.sv` and `.v` files.
3. All IP files are parsed together with a shared preprocessor context, so `` `define `` macros propagate across files.
4. When an instance references a module not defined in the `.slip` source, Slip uses slang to reflect the external module's ports and parameters.
5. Port widths are evaluated using the instance's parameter overrides.
6. Signals are created with correct widths for regex-mapped connections.

22.2 Filelist Support

Slip supports VCS-format filelist (`.f`) files for explicit control over IP source ordering, include paths, and macro definitions:

```
// ip.f
+incdir+./include
+define+DATA_WIDTH=32
+define+ADDR_WIDTH=8
./rtl/fifo.sv
./rtl/arbiter.sv
-f vendor_lib.f
```

- `+incdir+path` — include search directory (multiple paths separated by `+`)
- `+define+MACRO=value` — macro definition
- `-f sub.f` — nested filelist reference (recursive, with circular reference detection)
- File paths are resolved relative to the filelist location
- Duplicate files are automatically deduplicated
- Comments (`//`) and empty lines are ignored

22.3 Two-Pass Scanning

IP scanning uses a two-pass approach to resolve `` `define `` ordering:

Pass 1 — Collection: Parse all filelists, collect source files, include paths, and macro definitions. Recursively scan IP directories and merge with filelist entries (deduplication).

Pass 2 — Reflection: Parse all IP files together using `SyntaxTree.fromFiles()` with a shared preprocessor context. This ensures that `` `define `` macros in one file are available to subsequent files, solving the define ordering problem that would occur with per-file parsing.

22.4 Example

Given an external IP file `ip_cores/fifo.sv`:

```
module fifo #(
    parameter DEPTH = 16,
    parameter WIDTH = 8
)(
    input logic           clk,
    input logic           rst_n,
    input logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic          full
);
```

In your Slip source:

```
module top (clk, rst_n, data_in, data_out) {
    logic clk;
    logic rst_n;
    logic [7:0] data_in;
    logic [7:0] data_out;

    fifo #(.DEPTH(32)) u_fifo {
        .clk,
        .rst_n,
        .din(data_in),
        .dout(data_out),
        .full(_)
    };
}
```

The `.full(_)` connection leaves the `full` output dangling.

Chapter 23 Semantic Rules

23.1 Multi-Driver Detection

A signal cannot be driven by multiple procedural blocks. This is caught at compile time:

```
// ERROR: signal 'q' driven by multiple blocks
seq (clk) { q <= a; }
seq (clk) { q <= b; }
```

The analyzer tracks assignments through `if` and `for` nesting within each block. A signal driven by two `seq`/`comb` blocks, or by a procedural block and a continuous assignment, produces an error.

The check is bit-range aware: two continuous assignments driving *disjoint* bits of the same variable are legal, which is what unrolled code such as:

```
`for (`i = 0; `i< 4; `i= `i + 1) {
    assign mask[`i] = din[`i];
}
```

produces. Overlapping or whole-variable drivers are rejected.

23.2 Combinational Loop Detection

Feedback loops inside `comb` blocks are detected using DFS-based cycle detection on the signal dependency graph:

```
// ERROR: combinational loop detected
comb {
    a = b + 1;
    b = a + 1; // 'a' depends on 'b', 'b' depends on 'a'
}
```

The detector builds one dependency graph per module covering every `comb` block *and* every continuous assignment, so loops that cross a block boundary are caught as well.

Within a block, blocking (last-assignment-wins) semantics are modelled: a read resolves to the most recent prior assignment of what it reads, which is what keeps the standard *init-then-accumulate* idiom loop-free:

```
comb {
    y = 0;
    y = y + b;  // OK: the read resolves to the 0 just assigned
}
```

Graph nodes are (signal, index) pairs, so storage written in disjoint elements stays independent. A chain over array elements is fine, while a genuine element cycle is reported with its indices:

```
assign st[0] = din; // OK
assign st[1] = st[0];
assign st[2] = st[1];

// ERROR: combinational loop detected: st[2] -> st[1] -> st[0] -> st[2]
```

If a cycle is found, a `SlipSemanticError` is raised.

23.3 Width Mismatch Warnings

The compiler checks for width mismatches in assignments and instance port connections. When both sides have concrete integer widths and they differ, a warning is emitted:

```
module m (out[7:0], other[3:0]) {
    assign out = other; // WARNING: 'out' is 8 bits, but value is 4 bits
}
```

Unsized decimal literals are range-checked against the target rather than assumed to be 32 bits, so the common `y = 0;` idiom stays silent while a value that does not fit warns:

```
assign out = 42;  // OK: 42 fits in 8 bits
assign out = 300; // WARNING: value 300 does not fit in 'out' (8 bits)
```

Width-prefixed literals use their declared width:

```
module m (out[7:0]) {
    assign out = 8'hFF; // OK: both 8 bits
}
```

Instance port connections are also checked:

```
module sub (data[7:0]) { ... }
module top (out[3:0]) {
    sub u { .data(out) }; // WARNING: port expects 8 bits, signal is 4 bits
}
```

Parameterized widths with default values are evaluated using those defaults. Widths that cannot be evaluated are silently skipped.

23.4 Localparam Override Protection

Parameters can be overridden at instantiation; localparams cannot:

```
module example #(param W = 8) () {
    localparam MASK = 255;
}

// OK
example #(.W(16)) u1 {};
```

```
// ERROR: cannot override localparam 'MASK'
example #(.MASK(511)) u2 {};
```

23.5 Implicit Signal Width Inference

Implicit signals connected to instance ports automatically inherit the target port's width:

```
module top (a, b) {
    logic [7:0] a;
    logic [7:0] b;
    // 'sum' is implicit --- inherits [7:0] from adder.sum port
    adder u1 { .a, .b, .sum };
}
```

Generates:

```
    // width from adder.sum
    logic [7:0] sum;
```

Other implicit signals (not connected to ports) default to 1-bit and are annotated with `// implicit, no width` in the generated output.

23.6 Generated Code Annotations

Slip adds review annotations to generated SystemVerilog signals and ports:

Annotation	Meaning
`// implicit`	Signal or port was auto-generated (not explicitly declared)
`// no width`	Signal or port has no declared width (defaults to 1-bit)
`// width from X.Y`	Width was automatically inferred from instance port

These annotations help identify signals that may need explicit width declarations during code review. Use `grep "no width" *.sv` to quickly find all un-sized signals.

23.7 Integer Literal Validation

Verilog-style sized literals are validated at lex time:

- Digits must be valid for the specified radix (e.g., `4'bABCD` is rejected).
- The digit portion must not be empty or underscore-only (e.g., `4'b_` is rejected).
- Four-state values `x`/`z` are accepted in any radix.

Chapter 24 Error Reference

24.1 Compilation Errors

`cannot override localparam 'name' of module 'M' (instance 'I')`

Attempting to override a localparam in instance `#(...)`.

`unconnected input port 'name' in instance 'I' of module 'M'`

Input port not connected and not marked with `_`. The error includes active regex rules when applicable.

`unable to infer width for implicit signal 'name'`

Implicit signal used in a context where width cannot be determined (legacy; now defaults to 1-bit with `// implicit, no width` annotation).

`cyclic module dependency detected among: A, B`

Modules have circular instantiation dependency (e.g., module A instantiates B, and B instantiates A). This is illegal in synthesizable hardware.

`bare integer '...' in port connection must have explicit width`

Bare integer used in port connection. Use a width-prefixed literal (`1'b0`) or a fill constant (`'0`).

`signal 'name' driven by multiple drivers`

A signal is driven by more than one source (two procedural blocks, or a block and a continuous assignment) over overlapping bits. Disjoint bit-ranges are allowed.

`invalid digit 'X' for radix 'r' in literal`

Integer literal contains a digit invalid for the specified radix (e.g. `4'bABCD`).

`integer literal has no actual digit value`

Literal has only underscores after the radix prefix (e.g. `4'b_`).

`E_LIMIT`

A compilation resource limit was exceeded. Defaults include 10,000 metaprogram loop iterations and 100,000 expanded statements per module. Metaprogram controls and defun integers must fit a signed 64-bit integer; defun strings operate on bytes.

`cannot evaluate 'name' at compile time`

Non-constant expression in `` `for ``/`` `if `` condition, init, or step.

`` `for step variable '...' must match loop variable '...' ``

Step variable differs from the declared loop variable. The step expression determines the next value of the loop variable, so the step variable must match the loop variable.

`unsupported operator '...' in compile-time expression`

Operator not supported by the constant expression evaluator.

`combinational loop detected: ...`

A feedback loop was found. The message lists the storage involved, with indices where relevant (`st[2] -> st[1] -> st[0] -> st[2]`). The graph covers every `comb` block and every continuous assignment in the module.

`duplicate declaration of 'name'`

A signal, instance, or localparam with the same name is declared more than once in the same module.

duplicate module definition: 'name' is defined more than once

Two modules share a name in the compilation unit (including files pulled in by `include`).

duplicate connection to port 'name' of instance 'I'

The same target port is connected twice in one instance.

cannot resolve target module 'name' of instance 'I'

The instance target is neither defined in the compilation unit nor found in the IP index. Pass `-ip`/`-f` when the module is external IP; otherwise this is a typo.

nonblocking assignment '<=' is not allowed at module level

`<=` outside a `seq` block. Use a `seq (clk) { ... }` block for sequential logic, or `=` for a continuous assignment.

'if'/'for'/'case' is not supported at module level

A procedural statement at module level. Wrap it in `comb {}` or `seq (clk) {}`, or use the compile-time forms `` `if ``/`` `for ``.

empty concatenation '{}' is not allowed

A concatenation with no operands has no width and no SystemVerilog equivalent.

method '.name()' is not allowed in defun bodies

`defun` bodies are evaluated in a sandbox with a safe method allowlist.

regex port mapping error for port 'name' of instance 'I'

Replacement evaluation failed: unknown `$func`, invalid backreference, or bad arguments.

reset polarity inconsistency: ...

A reset signal is used with conflicting polarity (both `pos:` and `neg:`) across different `seq` blocks.

24.2 Warnings

in seq block of module 'M', the blocking assignment to 'name' reads its own previous value

A blocking assignment converts to nonblocking in a `seq` block (unconditionally, as described in the conversion chapter). This warning fires for the assignments whose meaning that changes: a read-modify-write of storage already written on the same path, such as a loop accumulator (`for (...) { sum = sum + b; }`) or `y = 0; y = y + b;`. The read will observe the pre-clock value, so restructure or use a `comb` block if the updated value was intended. Register transfers (shift registers, `t = a + 1; o = t;`) convert silently.

width mismatch: 'name' is N bits, but value is M bits

An assignment has mismatched widths between the target signal and the value expression.

value N does not fit in 'name' (M bits)

An unsized decimal literal is outside the target's range (e.g. `out = 300` for an 8-bit `out`).

port width mismatch: 'port' expects N bits, but 'signal' is M bits

An instance port connection has mismatched widths between the port declaration and the connected signal.

unconnected output port 'name' in instance 'I' of module 'M'

An output port matched no connection and no regex rule; it is left dangling.

duplicate module 'name' in IP sources

Two IP files define the same module. The first definition is kept; the choice is otherwise arbitrary.

`output directory ... contains SystemVerilog files not produced by this run`

Stale `.sv` files from an earlier build (a renamed module, say) are still present. Downstream tools that compile a whole directory would pick them up.

Chapter 25 Complete Examples

25.1 Parameterized Pipeline Register

```
module pipeline_reg #(param WIDTH = 8) (clk, rst_n, d, q) {
    logic clk;
    logic rst_n;
    logic [WIDTH-1:0] d;
    logic [WIDTH-1:0] q;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            q <= 0;
        } else {
            q <= d;
        }
    }
}
```

Generates:

```
module pipeline_reg #(
    parameter WIDTH = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH - 1:0] d,
    output logic [WIDTH - 1:0] q
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 0;
        end else begin
            q <= d;
        end
    end

endmodule
```

25.2 Bit Reversal with Metaprogramming

```
module bit_reverse #(param W = 8) (din, dout) {
    logic [W-1:0] din;
    logic [W-1:0] dout;

    comb {
```

```
        `for (`i = 0; `i< W; `i= `i + 1) {
            dout[W - 1 - `i] = din[`i];
        }
    }
}
```

Generates:

```
module bit_reverse #(
    parameter W = 8
) (
    input logic [W - 1:0] din,
    output logic [W - 1:0] dout
);

    if (W !== (8)) begin : __slip_frozen_W
        $error("W controls Slip compile-time expansion and cannot be overridden");
    end

    always_comb begin
        dout[W - 1 - 0] = din[0];
        dout[W - 1 - 1] = din[1];
        dout[W - 1 - 2] = din[2];
        dout[W - 1 - 3] = din[3];
        dout[W - 1 - 4] = din[4];
        dout[W - 1 - 5] = din[5];
        dout[W - 1 - 6] = din[6];
        dout[W - 1 - 7] = din[7];
    end

endmodule
```

Note: parameter-dependent indices remain symbolic. The downstream tool evaluates them during elaboration; W cannot be overridden because it controls compile-time expansion.

25.3 State Machine with Localparams

```
module arbiter (clk, rst_n, req, grant) {
    logic clk;
    logic rst_n;
    logic [3:0] req;
    logic [3:0] grant;
    logic [1:0] cur;

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            cur <= S_IDLE;
```

```
        } else {
            case (cur) {
                S_IDLE: {
                    if (req[0]) cur <= S_GNT0;
                    if (req[1]) cur <= S_GNT1;
                }
                S_GNT0: {
                    if (!req[0]) cur <= S_IDLE;
                }
                S_GNT1: {
                    if (!req[1]) cur <= S_IDLE;
                }
                default: cur <= S_IDLE;
            }
        }
    }

    assign grant[0] = (cur == S_GNT0);
    assign grant[1] = (cur == S_GNT1);
}
```

Generates:

```
module arbiter (
    input logic clk,
    input logic rst_n,
    input logic [3:0] req,
    output logic [3:0] grant
);

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;

    logic [1:0] cur;

    assign grant[0] = (cur == S_GNT0);
    assign grant[1] = (cur == S_GNT1);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cur <= S_IDLE;
        end else begin
            case (cur)
                S_IDLE: begin
                    if (req[0]) begin
                        cur <= S_GNT0;
                    end
                    if (req[1]) begin
                        cur <= S_GNT1;
                    end
                end
                S_GNT0: begin
```

```
                if (!req[0]) begin
                    cur <= S_IDLE;
                end
            end
            S_GNT1: begin
                if (!req[1]) begin
                    cur <= S_IDLE;
                end
            end
            default: begin
                cur <= S_IDLE;
            end
        endcase
    end
end

endmodule
```

25.4 Arbiter with Case Statement

```
module arbiter_case (clk, rst_n, req, grant) {
    logic clk;
    logic rst_n;
    logic [3:0] req;
    logic [3:0] grant;
    logic [2:0] cur;

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;
    localparam S_GNT2 = 3;
    localparam S_GNT3 = 4;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            cur <= S_IDLE;
        } else {
            case (cur) {
                S_IDLE: {
                    if (req[0]) cur <= S_GNT0;
                    if (req[1]) cur <= S_GNT1;
                    if (req[2]) cur <= S_GNT2;
                    if (req[3]) cur <= S_GNT3;
                }
                S_GNT0: {
                    if (!req[0]) cur <= S_IDLE;
                }
                S_GNT1: {
                    if (!req[1]) cur <= S_IDLE;
                }
```

```
                default: cur <= S_IDLE;
            }
        }
    }

    comb {
        grant = 0;
        case (cur) {
            S_GNT0: grant[0] = 1;
            S_GNT1: grant[1] = 1;
            S_GNT2: grant[2] = 1;
            S_GNT3: grant[3] = 1;
            default: grant = 0;
        }
    }
}
```

Generates:

```
module arbiter_case (
    input logic clk,
    input logic rst_n,
    input logic [3:0] req,
    output logic [3:0] grant
);

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;
    localparam S_GNT2 = 3;
    localparam S_GNT3 = 4;

    logic [2:0] cur;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cur <= S_IDLE;
        end else begin
            case (cur)
                S_IDLE: begin
                    if (req[0]) begin
                        cur <= S_GNT0;
                    end
                    if (req[1]) begin
                        cur <= S_GNT1;
                    end
                    if (req[2]) begin
                        cur <= S_GNT2;
                    end
                    if (req[3]) begin
                        cur <= S_GNT3;
                    end
                end
```

```
                S_GNT0: begin
                    if (!req[0]) begin
                        cur <= S_IDLE;
                    end
                end
                S_GNT1: begin
                    if (!req[1]) begin
                        cur <= S_IDLE;
                    end
                end
                default: begin
                    cur <= S_IDLE;
                end
            endcase
        end
    end

    always_comb begin
        grant = 0;
        case (cur)
            S_GNT0: begin
                grant[0] = 1;
            end
            S_GNT1: begin
                grant[1] = 1;
            end
            S_GNT2: begin
                grant[2] = 1;
            end
            S_GNT3: begin
                grant[3] = 1;
            end
            default: begin
                grant = 0;
            end
        endcase
    end

endmodule
```

25.5 Hierarchical Design

```
module stage #(param W = 8) (clk, rst_n, data_in, data_out) {
    logic clk;
    logic rst_n;
    logic [W-1:0] data_in;
    logic [W-1:0] data_out;
    logic [W-1:0] reg_data;

    seq (clk, neg: rst_n) {
```

```
        if (!rst_n) {
            reg_data <= 0;
        } else {
            reg_data <= data_in;
        }
    }

    assign data_out = reg_data;
}

module pipeline (clk, rst_n, din, dout) {
    logic clk;
    logic rst_n;
    logic [7:0] din;
    logic [7:0] dout;
    logic [7:0] stage_data [0:3];

    assign stage_data[0] = din;

    `for (`i = 1; `i< 3; `i= `i + 1) {
        stage #(.W(8)) u_stage_`i {
            .clk,
            .rst_n,
            .data_in(stage_data[`i - 1]),
            .data_out(stage_data[`i])
        };
    }

    assign dout = stage_data[2];
}
```

Generates:

```
module stage #(
    parameter W = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [W - 1:0] data_in,
    output logic [W - 1:0] data_out
);

    logic [W - 1:0] reg_data;

    assign data_out = reg_data;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            reg_data <= 0;
        end else begin
            reg_data <= data_in;
        end
    end
```

```
endmodule

module pipeline (
    input logic clk,
    input logic rst_n,
    input logic [7:0] din,
    output logic [7:0] dout
);

    logic [7:0] stage_data [0:3];

    assign stage_data[0] = din;
    assign dout = stage_data[2];

    stage #(
        .W(8)
    ) u_stage_1 (
        .clk(clk),
        .rst_n(rst_n),
        .data_in(stage_data[0]),
        .data_out(stage_data[1])
    );

    stage #(
        .W(8)
    ) u_stage_2 (
        .clk(clk),
        .rst_n(rst_n),
        .data_in(stage_data[1]),
        .data_out(stage_data[2])
    );

endmodule
```

Appendix A Formal Grammar (EBNF)

```
CompilationUnit ::= { TopLevelItem }

TopLevelItem ::= ModuleDecl | FuncDef | IncludeDirective

IncludeDirective ::= "include" STRING ";"

ModuleDecl ::= "module" IDENT
               [ "#(" ParameterList ")" ]
               [ "(" PortList ")" ]
               "{" { Statement } "}"

ParameterList ::= ParameterDecl { "," ParameterDecl }
ParameterDecl ::= "param" IDENT "=" Expr

PortList     ::= PortItem { "," PortItem }
PortItem     ::= IDENT [ "[" Expr [ ":" Expr ] "]" ]

Statement ::= SignalDecl
            | LocalparamDecl
            | AssignStmt
            | SeqBlock
            | CombBlock
            | InitialBlock
            | IfStmt
            | ForStmt
            | CaseStmt
            | GenForStmt
            | GenIfStmt
            | InstanceStmt
            | BlockStmt

SignalDecl ::= ["logic"] ["signed"] [ "[" Expr "]" ]
               IDENT [ "[" Expr ":" Expr "]" ] ";"

LocalparamDecl ::= "localparam" IDENT "=" Expr ";"

AssignStmt ::= ["assign"] Lvalue "=" Expr ";"
             | ["assign"] Lvalue CompoundOp Expr ";"

CompoundOp ::= "+=" | "-=" | "*=" | "/=" | "%="
             | "&=" | "|=" | "^=" | "<<=" | ">>="
             | "<<<=" | ">>>="

SeqBlock    ::= "seq" "(" IDENT
                [ "," ("pos" | "neg") ":" IDENT ] ")" BlockStmt
CombBlock   ::= "comb" BlockStmt
InitialBlock ::= "initial" BlockStmt

IfStmt ::= "if" "(" Expr ")" BlockStmt
```

```
          [ "else" BlockStmt ]

ForStmt ::= "for" "(" IDENT "=" Expr ";" Expr ";"
                ForStep ")" BlockStmt
ForStep ::= IDENT "=" Expr
         | IDENT CompoundOp Expr
         | IDENT "++"

CaseStmt ::= ("case" | "casez" | "casex") "(" Expr ")"
            "{" { CaseItem } "}"
CaseItem ::= PatternList ":" Statement
          | "default" ":" Statement
PatternList ::= Expr { "," Expr }

GenForStmt ::= "`for" "(" TICK_IDENT "=" Expr ";" Expr ";"
                       TICK_IDENT "=" Expr ")" BlockStmt
GenIfStmt ::= "`if" "(" Expr ")" BlockStmt
                      [ "`else" BlockStmt ]

InstanceStmt ::= IDENT
               [ "#(" NamedParam { "," NamedParam } ")" ]
               IDENT "{" { Connection } "}" [";"]

NamedParam ::= "." IDENT "(" Expr ")"
Connection ::= "." IDENT [ "(" Expr ")" ]
            | STRING "=>" STRING

BlockStmt ::= "{" { Statement } "}"
Lvalue   ::= IDENT { "[" Expr "]" | "[" Expr ":" Expr "]" }
```

Notes on the grammar:

- `TICK_IDENT` denotes a backtick-prefixed identifier (e.g., `` `i ``).
- Template identifiers (e.g., `data_`i`) are formed at the lexical level by merging adjacent `IDENT`, `TICK`, and `TICK_IDENT` tokens.
- Expressions are parsed by a Pratt parser and support all operators listed in the operator precedence table.
- Special expression forms include concatenation `{a, b}`, replication `{n{expr}}`, width cast `N'(expr)`, and set membership `x inside {v1, v2, ...}`.
- Compound assignment operators are desugared at parse time into regular assignments.